

6.3 Service Scoping

Part of Services and Dependency Injection Module - Duration: 10-12 hours

1. Understanding Service Scope

1.1 What is Service Scope?

Service scope defines the visibility and lifetime of a service instance within your Angular application. It determines **where** a service is available, **how many** instances exist, and **when** those instances are created and destroyed.

```
// Scope determines:  
1. WHERE can the service be used? (Which components/modules)  
2. HOW MANY instances exist? (One shared instance or multiple)  
3. WHEN is it created? (At app start, on-demand, or per component)  
4. WHEN is it destroyed? (With app, with module, or with component)
```

1.2 Why Does Scope Matter?

Understanding scope is crucial because it affects:

Concern	Impact of Scope
Data Sharing	Singleton services share data; separate instances don't
Memory Usage	Multiple instances consume more memory
Performance	Singleton creation happens once; per-component creation repeats
Bundle Size	Tree-shakable services can be removed if unused
Testing	Singleton state persists; isolated instances don't interfere

1.3 The Three Scopes

Angular provides three main scopes for services:

Root Scope (Application-Wide)

- └ One instance for entire application
- └ Available everywhere
- └ Recommended for most services

Module Scope (Feature-Specific)

- └ One instance per module
- └ Available within module and its components
- └ Used for feature-specific services

Component Scope (Local)

- └ New instance for each component
- └ Available only to component and its children
- └ Used when components need isolated state

2. Singleton Objects: The Root Scope

2.1 What is a Singleton?

A singleton is a design pattern that ensures only **one instance** of a class exists throughout the application's lifetime. Angular's root-scoped services are singletons.

Real-World Analogy

Think of a singleton like a city's central database:

- There's only ONE central database for the entire city
- All departments (police, hospital, school) access the SAME database
- When police update a record, hospital sees the same change
- No duplicate databases exist - one source of truth

2.2 Creating Singleton Services with `providedIn: 'root'`

The most common and recommended way to create a singleton service is using `providedIn: 'root'` :

```
// user.service.ts - Singleton Service
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Makes this a singleton
})
export class UserService {

  private currentUser: string = 'Guest';
  private loginCount: number = 0;

  constructor() {
    console.log('UserService created (happens only ONCE)');
  }

  setUser(name: string): void {
    this.currentUser = name;
    this.loginCount++;
  }

  getUser(): string {
    return this.currentUser;
  }

  getLoginCount(): number {
    return this.loginCount;
  }
}
```

Using the Singleton in Multiple Components

```
// header.component.ts
import { Component } from '@angular/core';
import { UserService } from './user.service';

@Component({
  selector: 'app-header',
  template: `
    <header>
      <h3>Header Component</h3>
      <p>User: {{ getUser() }}</p>
      <p>Login Count: {{ getLoginCount() }}</p>
      <button (click)="login()">Login as John</button>
    </header>`
})
```

```
`  
)  
export class HeaderComponent {  
  constructor(private userService: UserService) { }  
  
  getUser(): string {  
    return this.userService.getUser();  
  }  
  
  getLoginCount(): number {  
    return this.userService.getLoginCount();  
  }  
  
  login(): void {  
    this.userService.setUser('John Doe');  
  }  
}
```

```
// footer.component.ts  
import { Component } from '@angular/core';  
import { UserService } from '../user.service';  
  
@Component({  
  selector: 'app-footer',  
  template: `  
    <footer>  
      <h3>Footer Component</h3>  
      <p>User: {{ getUser() }}</p>  
      <p>Login Count: {{ getLoginCount() }}</p>  
      <button (click)="login()">Login as Jane</button>  
    </footer>  
  `,  
)  
export class FooterComponent {  
  constructor(private userService: UserService) { }  
  
  getUser(): string {  
    return this.userService.getUser();  
  }  
  
  getLoginCount(): number {  
    return this.userService.getLoginCount();  
  }  
  
  login(): void {
```

```
this.userService.setUser('Jane Smith');  
}  
}
```

Output:

Singleton Behavior:

Console Output (on app start):

UserService created (happens only ONCE)

When Header clicks "Login as John":

- Header shows: User: John Doe, Login Count: 1
- Footer ALSO shows: User: John Doe, Login Count: 1

When Footer clicks "Login as Jane":

- Footer shows: User: Jane Smith, Login Count: 2
- Header ALSO shows: User: Jane Smith, Login Count: 2

Key Point: Both components share the SAME UserService instance. Changes in one component are immediately visible in the other.

2.3 Characteristics of Singleton Services

Characteristic	Description	Benefit
Single Instance	Only one object exists in memory	Memory efficient, consistent state
Global Access	Available to all components/services	Easy data sharing across app
Lazy Creation	Created only when first injected	Faster app startup time
Lifetime	Lives for entire application lifetime	State persists throughout app

Characteristic	Description	Benefit
Shared State	All consumers see same data	Centralized data management

2.4 When to Use Singleton Services

Use singleton services (providedIn: 'root') when you need:

- **Shared Data:** Application state, user information, configuration
- **Centralized Logic:** Authentication, logging, data transformation
- **Caching:** Store data fetched from server to avoid repeated calls
- **Event Broadcasting:** Notify multiple components of changes
- **Utilities:** Helper functions, calculations, formatters

```
// Perfect candidates for singleton services:  
  
@Injectable({ providedIn: 'root' })  
export class AuthService { } // Authentication state  
  
@Injectable({ providedIn: 'root' })  
export class ConfigService { } // App configuration  
  
@Injectable({ providedIn: 'root' })  
export class LoggerService { } // Centralized logging  
  
@Injectable({ providedIn: 'root' })  
export class CacheService { } // Data caching
```

3. Module-Level Scope

3.1 What is Module-Level Scope?

Module-level scoped services create one instance per module. If multiple modules provide the same service, each module gets its own instance.

3.2 Creating Module-Scoped Services

There are two ways to provide services at module level:

Method 1: Using providers Array in @NgModule

```
// feature.service.ts
import { Injectable } from '@angular/core';

@Injectable() // No providedIn
export class FeatureService {

  private featureData: string = 'Default';

  constructor() {
    console.log('FeatureService created');
  }

  setData(data: string): void {
    this.featureData = data;
  }

  getData(): string {
    return this.featureData;
  }
}
```

```
// feature.module.ts
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FeatureComponent } from './feature.component';
import { FeatureService } from './feature.service';

@NgModule({
  declarations: [FeatureComponent],
  imports: [CommonModule],
  providers: [FeatureService] // Provided at module level
})
export class FeatureModule { }
```

Method 2: Using providedIn with Module Reference

```
// another-feature.service.ts
import { Injectable } from '@angular/core';
import { AnotherFeatureModule } from './another-feature.module';

@Injectable({
  providedIn: AnotherFeatureModule // Scoped to specific module
})
export class AnotherFeatureService {
  // Service code
}
```

Output:

Module-Level Behavior:

- Each module that provides the service gets its own instance
- Components within the module share that module's instance
- Different modules have different instances (no sharing between modules)
- Service is not tree-shakable (stays in bundle even if unused)

3.3 Module-Level vs Root-Level Comparison

```
// Scenario: Two feature modules both use the same service
```

```
// With providedIn: 'root'
```

```
Module A Component 1 └─┐
Module A Component 2 └─┐→ Same Service Instance
Module B Component 1 └─┐
Module B Component 2 └─┐
```

```
// With module-level providers
```

```
Module A Component 1 └─┐→ Service Instance A
Module A Component 2 └─┐
```

```
Module B Component 1 └─┐→ Service Instance B (different!)
Module B Component 2 └─┐
```


3.4 Practical Example: Module-Scoped Service

```
// admin-state.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class AdminStateService {

  private adminMode: boolean = false;
  private viewType: string = 'grid';

  constructor() {
    console.log('AdminStateService created for module');
  }

  setAdminMode(mode: boolean): void {
    this.adminMode = mode;
  }

  isAdminMode(): boolean {
    return this.adminMode;
  }

  setViewType(type: string): void {
    this.viewType = type;
  }

  getViewType(): string {
    return this.viewType;
  }
}
```

```
// admin.module.ts
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AdminDashboardComponent } from './admin-dashboard.component';
import { AdminUsersComponent } from './admin-users.component';
import { AdminStateService } from './admin-state.service';

@NgModule({
  declarations: [
    AdminDashboardComponent,
    AdminUsersComponent
  ],
```

```
imports: [CommonModule],
providers: [AdminStateService] // Only admin module gets this instance
})
export class AdminModule { }
```

Output:

Result:

- AdminDashboardComponent and AdminUsersComponent share the same AdminStateService instance
- Other modules cannot access this instance
- If another module provides AdminStateService, it gets a different instance
- Admin module's state is isolated from the rest of the application

3.5 When to Use Module-Level Scope

Use module-level scope when you need:

- **Feature Isolation:** State specific to one feature module
- **Module-Specific Configuration:** Settings that differ per module
- **Legacy Compatibility:** Older Angular patterns before providedIn
- **Separate Instances:** Each lazy-loaded module needs its own service instance

⚡ Important:

Drawback: Module-level services are NOT tree-shakable. They're included in the bundle even if never used. Prefer `providedIn: 'root'` unless you specifically need module-level scoping.

4. Singleton vs Regular Object

4.1 What Makes a Service a Singleton?

A service becomes a singleton based on **where** and **how** it's provided:

```
// SINGLETON: providedIn: 'root'
@Injectable({
  providedIn: 'root'
})
export class SingletonService { }
// → One instance for entire application

// NOT SINGLETON: Provided in component
@Component({
  providers: [NonSingletonService]
})
export class MyComponent { }
// → New instance for each component
```

4.2 Demonstrating Singleton vs Multiple Instances

Let's create a counter service to clearly show the difference:

```
// counter.service.ts
import { Injectable } from '@angular/core';

@Injectable() // No providedIn - we'll provide it differently
export class CounterService {

  private count: number = 0;
  private instanceId: number;

  constructor() {
    this.instanceId = Math.floor(Math.random() * 1000);
    console.log(`CounterService instance ${this.instanceId} created`);
  }

  increment(): void {
    this.count++;
  }

  getCount(): number {
    return this.count;
  }

  getInstanceId(): number {
```

```
    return this.instanceId;
  }
}
```

Scenario 1: Singleton (Root-Level)

```
// app.module.ts
@NgModule({
  providers: [CounterService] // Provided at root
})
export class AppModule { }
```

```
// component-a.component.ts
@Component({
  selector: 'app-component-a',
  template: `
    <div>
      <h3>Component A</h3>
      <p>Instance ID: {{ getInstanceId() }}</p>
      <p>Count: {{ getCount() }}</p>
      <button (click)="increment()">Increment</button>
    </div>
  `,
  // NO providers array
})
export class ComponentAComponent {
  constructor(private counter: CounterService) { }

  increment() { this.counter.increment(); }
  getCount() { return this.counter.getCount(); }
  getInstanceId() { return this.counter.getInstanceId(); }
}
```

```
// component-b.component.ts
@Component({
  selector: 'app-component-b',
  template: `
    <div>
      <h3>Component B</h3>
      <p>Instance ID: {{ getInstanceId() }}</p>
      <p>Count: {{ getCount() }}</p>
    </div>
  `
})
export class ComponentBComponent {
  constructor(private counter: CounterService) { }
```

```
    <button (click)="increment()">Increment</button>
  </div>
  `
  // NO providers array
})
export class ComponentBComponent {
  constructor(private counter: CounterService) { }

  increment() { this.counter.increment(); }
  getCount() { return this.counter.getCount(); }
  getInstanceId() { return this.counter.getInstanceId(); }
}
```

Output:

Singleton Output:

Console:

CounterService instance 742 created

Component A Display:

Instance ID: 742

Count: 0

Component B Display:

Instance ID: 742 (same!)

Count: 0

After Component A clicks increment:

Component A Count: 1

Component B Count: 1 (both see the change!)

Scenario 2: Multiple Instances (Component-Level)

```
// component-a.component.ts
@Component({
  selector: 'app-component-a',
  template: `
```

```
<div>
  <h3>Component A</h3>
  <p>Instance ID: {{ getInstanceId() }}</p>
  <p>Count: {{ getCount() }}</p>
  <button (click)="increment()">Increment</button>
</div>
`,
providers: [CounterService] // Provided at component level
})
export class ComponentAComponent {
  constructor(private counter: CounterService) { }

  increment() { this.counter.increment(); }
  getCount() { return this.counter.getCount(); }
  getInstanceId() { return this.counter.getInstanceId(); }
}
```

```
// component-b.component.ts
@Component({
  selector: 'app-component-b',
  template: `
    <div>
      <h3>Component B</h3>
      <p>Instance ID: {{ getInstanceId() }}</p>
      <p>Count: {{ getCount() }}</p>
      <button (click)="increment()">Increment</button>
    </div>
  `,
  providers: [CounterService] // Provided at component level
})
export class ComponentBComponent {
  constructor(private counter: CounterService) { }

  increment() { this.counter.increment(); }
  getCount() { return this.counter.getCount(); }
  getInstanceId() { return this.counter.getInstanceId(); }
}
```

Output:

Multiple Instances Output:

Console:

CounterService instance 742 created
 CounterService instance 315 created

Component A Display:

Instance ID: 742
 Count: 0

Component B Display:

Instance ID: 315 (different!)
 Count: 0

After Component A clicks increment:

Component A Count: 1
 Component B Count: 0 (no change - separate instance!)

4.3 Singleton vs Multiple Instances Comparison

Aspect	Singleton (Root/Module)	Multiple Instances (Component)
Instance Count	One	One per component
State Sharing	All components see same data	Each component has isolated data
Memory Usage	Lower (one object)	Higher (multiple objects)
Creation Time	Once, when first needed	Every component instantiation
Destruction	When app closes	When component destroys
Use Case	Shared data/logic	Component-specific state

5. Tree-Shakability

5.1 What is Tree-Shaking?

Tree-shaking is a process where unused code is removed from the final production bundle. It's like shaking a tree to remove dead leaves - only the live code remains.

```
// Before Tree-Shaking (Development)
Your App Bundle
├─ Service A (used) ✓
├─ Service B (used) ✓
├─ Service C (NOT used) ✗
├─ Service D (NOT used) ✗
└─ Service E (used) ✓

// After Tree-Shaking (Production)
Your App Bundle
├─ Service A (used) ✓
├─ Service B (used) ✓
└─ Service E (used) ✓

→ Services C and D removed → Smaller bundle!
```

5.2 How Tree-Shakability Works with Services

Only services with `providedIn` in the decorator are tree-shakable:

```
// TREE-SHAKABLE ✓
@Injectable({
  providedIn: 'root' // Tree-shakable
})
export class TreeShakableService {
  // If never injected, removed from bundle
}
```

```
// NOT TREE-SHAKABLE ✗
@Injectable()
export class NotTreeShakableService { }
```



```
// app.module.ts
@NgModule({
  providers: [NotTreeShakableService] // Always in bundle
})
```

5.3 Why Tree-Shakability Matters

Benefit	Impact
Smaller Bundle Size	Unused services removed → Faster downloads
Faster Load Time	Less JavaScript to parse → Quicker startup
Better Performance	Less code to execute → Smoother app
Automatic Optimization	Angular CLI handles it → No manual work

5.4 Practical Example of Tree-Shaking

```
// analytics.service.ts - Tree-shakable
@Injectable({
  providedIn: 'root'
})
export class AnalyticsService {
  trackEvent(event: string) {
    console.log('Event tracked:', event);
  }
}
```

```
// debug.service.ts - Tree-shakable
@Injectable({
  providedIn: 'root'
})
export class DebugService {
  logDebug(message: string) {
    console.log('DEBUG:', message);
  }
}
```

```
// app.component.ts
import { Component } from '@angular/core';
import { AnalyticsService } from '../analytics.service';
// DebugService NOT imported or used

@Component({
  selector: 'app-root',
  template: '<h1>My App</h1>'
})
export class AppComponent {
  // Only AnalyticsService injected
  constructor(private analytics: AnalyticsService) {
    this.analytics.trackEvent('App Loaded');
  }
}
```

Output:

Production Build Result:

- `AnalyticsService` → Included in bundle (used) ✓
- `DebugService` → Removed from bundle (never injected) ✗
- Result: Smaller bundle, faster app

5.5 Tree-Shakability Best Practices

💡 Tip:

Best Practices:

1. **Always use `providedIn`:** Prefer `providedIn: 'root'` over module providers
2. **Avoid eager loading:** Don't inject services in constructors unless needed
3. **Lazy load features:** Feature modules are tree-shakable units
4. **Check bundle size:** Use `ng build --stats-json` to analyze
5. **Remove unused imports:** Delete service imports if not used

6. Registering Services: Comparison

6.1 Three Ways to Register Services

Angular provides three methods to register services, each with different characteristics:

Method 1: @Injectable with providedIn (Recommended)

```
// Method 1: @Injectable with providedIn
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Registration in decorator
})
export class ModernService {
  getData() {
    return 'Modern approach';
  }
}
```

Output:

Characteristics:

- ✓ Tree-shakable
- ✓ Singleton by default
- ✓ Self-registering (no module config needed)
- ✓ Lazy-created (only when first injected)
- ✓ Recommended by Angular team

Method 2: Module providers Array

```
// Method 2: Module providers array
import { Injectable } from '@angular/core';

@Injectable() // No providedIn
export class LegacyService {
  getData() {
```

```
    return 'Legacy approach';
  }
}
```

```
// app.module.ts
import { NgModule } from '@angular/core';
import { LegacyService } from '../legacy.service';

@NgModule({
  providers: [LegacyService] // Registration in module
})
export class AppModule { }
```

Output:

Characteristics:

- ✗ Not tree-shakable
- ✓ Singleton at module level
- ✗ Requires module configuration
- ✓ Eagerly created with module
- ⚠ Older Angular pattern

Method 3: Component providers Array

```
// Method 3: Component providers array
import { Injectable } from '@angular/core';

@Injectable() // No providedIn
export class LocalService {
  getData() {
    return 'Local to component';
  }
}
```

```
// my.component.ts
import { Component } from '@angular/core';
import { LocalService } from '../local.service';
```

```
@Component({
  selector: 'app-my',
  template: '<div>My Component</div>',
  providers: [LocalService] // Registration in component
})
export class MyComponent {
  constructor(private local: LocalService) { }
}
```

Output:

Characteristics:

- ✗ Not tree-shakable
- ✗ New instance per component
- ✗ Requires component configuration
- ✓ Created with component
- ⚠ Use only when isolation needed

6.2 Complete Comparison Table

Feature	@Injectable providedIn	Module providers	Component providers
Tree-Shakable	✓ Yes	✗ No	✗ No
Instance Count	One (singleton)	One per module	One per component
Configuration Location	Service file	Module file	Component file
Creation Timing	Lazy (on demand)	With module	With component
State Sharing	App-wide	Module-wide	Component-only
Recommended Use	Most services	Legacy code	Isolated state

6.3 Migration Example: Legacy to Modern

Converting from module providers to providedIn:

```
// BEFORE: Legacy approach
// data.service.ts
@Injectable()
export class DataService { }

// app.module.ts
@NgModule({
  providers: [DataService]
})
```

```
// AFTER: Modern approach
// data.service.ts
@Injectable({
  providedIn: 'root' // Add this
})
export class DataService { }

// app.module.ts
@NgModule({
  providers: [] // Remove DataService
})
```

⚡ Important:

Migration Benefits:

- Service becomes tree-shakable
- No module configuration needed
- Follows Angular best practices
- Easier to test and maintain

7. Practical Decision Guide

7.1 Which Scope Should I Use?

```
Start Here: Do you need to share data across components?
|
└─ YES → Should it be available everywhere?
```

```

|
| └ YES → Use providedIn: 'root'
|       ✓ Most common choice
|       ✓ Singleton
|       ✓ Tree-shakable
|
| └ NO → Only within a feature?
|       |
|       └ YES → Use module providers
|             ⚠ Module-scoped
|             ⚠ Not tree-shakable
|
| └ NO → Each component needs its own?
|       |
|       └ YES → Use component providers
|             ⚠ Multiple instances
|             ⚠ Not tree-shakable

```

7.2 Common Use Cases

```

// Use Case 1: Application-wide data
@Injectable({ providedIn: 'root' })
export class UserService { } // User info needed everywhere

// Use Case 2: Feature-specific state
@NgModule({ providers: [FeatureStateService] }) // Only for this feature

// Use Case 3: Component-local state
@Component({ providers: [FormStateService] }) // Each form independent

```

7.3 Real-World Example: E-commerce App

```

// Root-level services (shared across app)
@Injectable({ providedIn: 'root' })
export class CartService { } // Shopping cart state

@Injectable({ providedIn: 'root' })
export class AuthService { } // User authentication

@Injectable({ providedIn: 'root' })
export class ProductService { } // Product catalog

```

```
// Module-level service (admin feature only)
@NgModule({ providers: [AdminDashboardService] })
export class AdminModule { }

// Component-level service (each product form independent)
@Component({ providers: [ProductFormService] })
export class ProductFormComponent { }
```

8. Summary

⚡ Important:

Key Takeaways:

1. Service Scope:

- Determines where services are available and how many instances exist
- Three main scopes: Root, Module, Component
- Affects data sharing, memory usage, and performance

2. Singleton Services:

- One instance for entire application
- Created with `providedIn: 'root'`
- Share state across all components
- Recommended for most services

3. Module-Level Services:

- One instance per module
- Provided in module's providers array
- Not tree-shakable
- Use for feature-specific services

4. Component-Level Services:

- New instance for each component
- Provided in component's providers array
- Isolated state per component
- Use when components need separate instances

5. Tree-Shakability:

- Only `providedIn` services are tree-shakable
- Unused services removed from production bundle
- Results in smaller, faster applications
- Always prefer `providedIn: 'root'`

6. Best Practice:

- Default to `providedIn: 'root'`
- Use module/component providers only when necessary
- Favor tree-shakability for better performance

9. Practice Exercise

Note:

Exercise: Shopping Cart with Different Scopes

Build a shopping application that demonstrates different service scopes:

1. Root-Level Service (Singleton):

Create `ProductCatalogService` with `providedIn: 'root'`:

- Store array of products (id, name, price)
- Method: `getAllProducts()`
- Method: `getProductById(id)`
- Initialize with 5 sample products

2. Root-Level Singleton:

Create `GlobalCartService` with `providedIn: 'root'`:

- Depends on `ProductCatalogService`
- Method: `addToCart(productId)`
- Method: `getCartItems()`
- Method: `getCartTotal()`
- Property: `cartCount`

3. Component-Level Service:

Create `WishlistService` (no providedIn):

- Provide it at component level
- Method: `addToWishlist(productId)`
- Method: `getWishlist()`
- Each component should have its own wishlist

4. Components:

- `ProductListComponent` - Display products, inject `ProductCatalogService` and `GlobalCartService`
- `CartComponent` - Show cart contents, inject `GlobalCartService`
- `WishlistComponent` - Show wishlist, provide `WishlistService` at component level

5. Test Requirements:

- Add products to cart from `ProductListComponent`
- Verify `CartComponent` shows same items (singleton)
- Create two `WishlistComponents`
- Verify each has independent wishlist (separate instances)
- Log instance creation in constructors to prove singleton vs multiple instances

Next: 6.4 Advanced Dependency Injection - Factory providers, optional injection, and multi-provider tokens